PROJECT REPORT

AI IT HELPDESK AGENT

1. PROJECT OVERVIEW

1.1 Project Title

AI IT Helpdesk Agent Using Agentic AI, Retrieval Augmented Generation, Tools and Memory

The project is an Artificial Intelligence based IT Helpdesk Agent designed to provide quick and reliable solutions for common computer and technical problems. The system uses Agentic AI concepts, Retrieval Augmented Generation (RAG), document retrieval, safe diagnostic tools, conversation memory, and a Large Language Model.

The main purpose of this project is to reduce the time users spend searching for solutions to common IT problems and contacting technical support for basic troubleshooting.

The system allows users to describe their technical problem using natural language. For example, a user can enter:

- “My Wi-Fi is connected but there is no internet.”
- “My laptop is very slow.”
- “My printer is not printing.”
- “Bluetooth is not connecting.”
- “How can I check my IP address?”

The AI IT Helpdesk Agent understands the user’s problem, identifies the technical category, retrieves relevant troubleshooting information from the knowledge base, performs safe system diagnostics when required, and generates a clear troubleshooting response.

1.2 Problem Statement

Users regularly face common IT problems such as Wi-Fi connectivity issues, internet problems, DNS errors, slow computers, printer failures, Bluetooth connection problems, software errors, Windows issues, and network problems.

In a traditional IT support environment, users may need to contact a helpdesk technician or search through multiple websites, manuals, FAQs, and support documents to find the correct solution.

When the problem is simple, contacting technical support can take unnecessary time. Searching manually can also be difficult because users may not know the correct technical terms.

For example, a user may want to know:

- Why is my Wi-Fi connected but there is no internet?
- Why is my laptop very slow?
- Why is my printer not printing?
- Why is Bluetooth not connecting?
- How can I check my IP address?
- Why is DNS not working?
- Why is an application not opening?
- How can I check my computer performance?
- Why is my network connection unstable?
- Why am I unable to log in?

A normal chatbot may answer these questions using general knowledge. However, general AI knowledge may not always contain the required troubleshooting information or may provide unsupported suggestions.

Therefore, a system is required that can understand technical problems and provide troubleshooting solutions based on a dedicated IT knowledge base.

The proposed **AI IT Helpdesk Agent** solves this problem by combining Artificial Intelligence with a troubleshooting knowledge base and safe system diagnostic tools.

The system retrieves relevant troubleshooting information before generating the final response. It can also execute predefined diagnostic functions such as checking the local IP address, internet connectivity, DNS resolution, CPU usage, memory usage, and disk usage.

Thus, the main problem addressed by this project is:

“How can an AI-based helpdesk system understand common IT problems, retrieve reliable troubleshooting information, perform safe system

diagnostics, and provide clear solutions while maintaining conversation context?"

1.3 Brief Description of the Project

The **AI IT Helpdesk Agent** is a web-based AI application developed to help users solve common computer and technical problems through natural language questions.

The user interacts with the system through a simple chat-based interface. The user does not need to know technical commands. They can simply describe the problem in normal language.

For example:

User:
"My laptop is very slow."

The question is received by the AI Helpdesk Agent and processed using different components of the system.

The project uses **Retrieval Augmented Generation (RAG)** to connect the AI model with a local IT troubleshooting knowledge base.

The knowledge base contains documents related to:

- Wi-Fi
- Internet
- Windows
- Printer
- Bluetooth
- Software
- Network
- Performance
- Login
- DNS

These documents are processed and divided into smaller text chunks. Each chunk is converted into an embedding and stored in a vector database.

When the user asks a question, the RAG system searches the vector database and retrieves the most relevant troubleshooting information.

The retrieved information is then provided to the AI model. The model uses the retrieved context to generate a clear and useful troubleshooting response.

The system also contains predefined safe diagnostic tools such as:

1. System Information Tool
2. Hostname Tool
3. Local IP Tool
4. Internet Connection Check
5. DNS Resolution Check
6. Public Host Ping
7. CPU Usage Check
8. Memory Usage Check
9. Disk Usage Check

These tools help the Agent collect useful diagnostic information without executing arbitrary system commands.

Another important component is **Conversation Memory**. The system maintains recent conversation context so that the Agent can understand follow-up questions.

For example:

User:
“My laptop is very slow.”

AI:
“Your problem appears to be related to system performance.”

User:
“How can I check my memory usage?”

The system can use the previous conversation context to understand what the user is asking.

The system also stores questions, responses, categories, timestamps, and feedback in an SQLite database.

Therefore, the project is not just a basic chatbot. It combines:

- AI Agent
- RAG
- Vector Database
- Knowledge Base

- Tool Calling
- System Diagnostics
- Conversation Memory
- SQLite Database
- Feedback
- Analytics

to create an intelligent IT support system.

2. OBJECTIVES & PROPOSED SOLUTION

2.1 Project Objectives

The primary objective of the project is to develop an AI-powered IT Helpdesk Agent that can understand common technical problems and provide useful troubleshooting solutions.

The major objectives are given below.

1. Provide Quick IT Support

The system should allow users to obtain troubleshooting assistance without manually searching multiple websites or documents.

2. Understand Natural Language Problems

Users should be able to describe their technical problem in normal language without using technical commands.

3. Implement Retrieval Augmented Generation

RAG should be used to retrieve relevant troubleshooting information before generating an answer.

4. Improve Answer Reliability

The AI should use retrieved knowledge base information instead of depending only on general knowledge.

5. Implement Semantic Search

The system should search the knowledge base based on the meaning of the user's question.

6. Classify IT Problems

The Agent should identify the appropriate category such as:

- Wi-Fi
- Internet
- Windows
- Printer
- Bluetooth
- Software
- Network
- Performance
- Login
- DNS
- General IT

7. Implement Safe Diagnostic Tools

The Agent should be able to use predefined diagnostic tools when additional system information is required.

8. Provide Clear Troubleshooting Steps

The system should provide simple step-by-step instructions that users can easily understand.

9. Maintain Conversation Context

The system should remember recent messages and understand follow-up questions.

10. Store Chat History

The system should store user questions, AI responses, categories, timestamps, and feedback.

11. Provide Feedback

Users should be able to indicate whether the generated answer was helpful or not helpful.

12. Provide Statistics

The system should display useful statistics such as total questions, feedback, and common problem categories.

13. Provide Demo Mode

The application should continue working with predefined troubleshooting responses when an external LLM API key is unavailable.

14. Protect System Security

The application should only execute predefined safe diagnostic functions and should not execute arbitrary shell or PowerShell commands.

15. Create a Foundation for an Advanced IT Assistant

The project should provide a foundation that can later be extended with additional diagnostic tools, authentication, multilingual support, voice interaction, and advanced AI capabilities.

2.2 How the Agentic AI Solution Works

The proposed solution combines multiple components to create an intelligent IT troubleshooting workflow.

OVERALL WORKING FLOW:

User Problem → Streamlit UI → AI Agent → Problem Classification → RAG Retrieval → Tool Decision → Safe Diagnostics → AI Model → Final Troubleshooting Response → Memory → SQLite → Feedback

The process is explained below.

Step 1 – User Enters a Technical Problem

The user opens the AI IT Helpdesk Agent and enters a technical problem.

Example:

“My Wi-Fi is connected but there is no internet.”

Step 2 – Frontend Sends the Question

The Streamlit interface receives the user’s question and sends it to the backend Agent logic.

Step 3 – Agent Receives the Question

The AI Agent receives the user's message and validates the input.

Empty or invalid questions are handled properly.

Step 4 – Conversation Memory is Checked

The system checks recent conversation history.

This helps the Agent understand follow-up questions and maintain context.

Step 5 – Problem Category is Identified

The system identifies the most relevant technical category.

For example:

Wi-Fi Problem → Wi-Fi Category

Slow Laptop → Performance Category

Printer Problem → Printer Category

Step 6 – Relevant Information is Retrieved

The RAG system searches the vector database.

The user's question is converted into an embedding and compared with the embeddings stored in the knowledge base.

The most relevant troubleshooting chunks are retrieved.

Step 7 – Tool Requirement is Decided

The Agent checks whether a diagnostic tool is required.

For example:

"How can I check my IP address?"

→ Local IP diagnostic tool.

"Is my internet working?"

→ Internet connection check.

"Why is my laptop slow?"

→ CPU, memory, and disk diagnostic tools may be useful.

Step 8 – Safe Diagnostic Tool is Executed

If required, the system executes only predefined safe diagnostic functions.

The project does not allow arbitrary shell or PowerShell execution.

Step 9 – Information is Combined

The following information is combined:

- User question
- Problem category
- Retrieved RAG information
- Diagnostic results
- Conversation history

Step 10 – AI Generates the Response

The AI model uses the available information to generate a clear troubleshooting response.

The response is organized into sections such as:

- Problem Identified
- Possible Causes
- Troubleshooting Steps
- Diagnostic Information
- Warning
- Recommended Solution
- Follow-up Question

Step 11 – Response is Displayed

The final response is displayed to the user through the Streamlit chat interface.

Step 12 – Feedback is Collected

The user can provide feedback such as:

- Helpful
- Not Helpful

Step 13 – Information is Stored

The question, response, category, timestamp, and feedback are stored in the SQLite database.

2.3 Key Features

The AI IT Helpdesk Agent provides several important features.

1. AI-Based Troubleshooting

The system uses AI to understand technical problems and generate useful troubleshooting responses.

2. Retrieval Augmented Generation

The system retrieves relevant information from the local IT knowledge base before generating answers.

3. IT Knowledge Base

The system contains troubleshooting documents for common technical categories.

4. Semantic Search

The system retrieves information based on the meaning of the user's question.

5. AI Agent

The Agent analyzes the problem, selects relevant information, decides whether tools are required, and generates the final response.

6. Safe Tool Calling

The system provides predefined diagnostic tools for collecting system information.

7. System Diagnostics

The application can check:

- Operating System
- Hostname
- Local IP
- CPU Usage
- Memory Usage
- Disk Usage

- Internet Connection
- DNS Resolution
- Network Connectivity

8. Conversation Memory

The system remembers recent messages and supports follow-up questions.

9. SQLite Database

Chat history and feedback are stored using SQLite.

10. Feedback System

Users can indicate whether the response was helpful.

11. Statistics Dashboard

The system displays:

- Total Questions
- Helpful Responses
- Not Helpful Responses
- Common Categories
- Usage Statistics

12. Knowledge Base Management

The system allows safe text documents to be viewed, added, deleted, and re-indexed.

13. Demo Mode

The application can work using predefined responses when the LLM API key is unavailable.

14. Error Handling

The application handles:

- Empty input
- Missing API key
- Invalid API configuration
- RAG database errors
- Tool failures

- Database errors
- LLM errors
- Permission errors

15. Student-Friendly Interface

The Streamlit interface provides a simple ChatGPT-style experience.

16. Modular Architecture

The system separates different responsibilities into modules such as:

- Agent
- RAG
- Tools
- Memory
- Database
- Knowledge Base
- User Interface
- Diagnostics
- Statistics

3. IMPLEMENTATION & RESULTS

3.1 Technologies / Tools Used

The project uses several technologies to implement the AI IT Helpdesk Agent.

Python

Python is the main programming language used for AI processing, RAG, system diagnostics, database operations, and application development.

- Artificial Intelligence
- Machine Learning
- Natural Language Processing
- System Monitoring
- Database Integration
- File Processing
- Web Application Development

Streamlit

Streamlit is used to create the interactive web interface.

It provides:

- Chat interface
- Buttons
- Dashboard
- Tables
- Charts
- Navigation
- User interaction

Large Language Model API

A configurable LLM API is used to generate natural-language troubleshooting responses.

The API key is stored securely using environment variables.

Sentence Transformers

Sentence Transformers are used to convert knowledge base text and user questions into numerical embeddings.

These embeddings are used for semantic similarity search.

ChromaDB

ChromaDB is used as the vector database.

It stores document embeddings and supports similarity-based retrieval.

SQLite

SQLite is used to store:

- User questions
- AI responses
- Categories
- Timestamps
- Feedback
- Session information

python-dotenv

python-dotenv is used to load configuration values such as the LLM API key from the .env file.

psutil

psutil is used for safe system diagnostics such as:

- CPU usage
- Memory usage
- Disk usage

socket

The socket library is used for:

- Hostname
- Local IP
- DNS-related operations

platform

The platform library is used to obtain operating system and system information.

subprocess

Subprocess is used only for predefined safe diagnostic operations when required.

Arbitrary commands are not accepted from users.

Visual Studio Code

Visual Studio Code can be used as the main development environment for writing and testing the project.

3.2 Working Process

The working process of the project can be divided into several stages.

Stage 1 – Collecting Knowledge Base Documents

Troubleshooting documents are added to the project knowledge base.

The supported knowledge base files include:

- wifi.txt
- internet.txt
- windows.txt
- printer.txt
- bluetooth.txt
- software.txt
- network.txt
- performance.txt
- login.txt
- dns.txt

These documents contain troubleshooting information.

Stage 2 – Reading Documents

The document loader reads the knowledge base files.

Stage 3 – Cleaning Text

The extracted text is cleaned to remove unnecessary spaces and formatting problems.

Stage 4 – Splitting Text

Large documents are divided into smaller text chunks.

Chunking allows the RAG system to retrieve only the relevant portions of a document.

Stage 5 – Creating Embeddings

Each text chunk is converted into a numerical vector called an embedding.

Stage 6 – Storing Embeddings

The embeddings and document metadata are stored in ChromaDB.

Metadata may contain:

- Filename
- Category
- Document information
- Chunk information

Stage 7 – User Asks a Question

The user enters a technical problem through the Streamlit interface.

Stage 8 – Problem Classification

The Agent identifies the most suitable technical category.

Stage 9 – RAG Retrieval

The question is converted into an embedding.

The vector database searches for semantically similar knowledge base chunks.

Stage 10 – Tool Processing

If additional system information is required, the Agent calls an appropriate predefined diagnostic tool.

Stage 11 – Memory Processing

Recent conversation history is retrieved from the conversation memory.

Stage 12 – AI Processing

The AI model receives:

- User question
- Problem category
- Retrieved knowledge
- Diagnostic results
- Conversation context

Stage 13 – Answer Generation

The AI generates the final troubleshooting response.

Stage 14 – Feedback and Database Storage

The question, response, category, timestamp, and feedback are stored in SQLite.

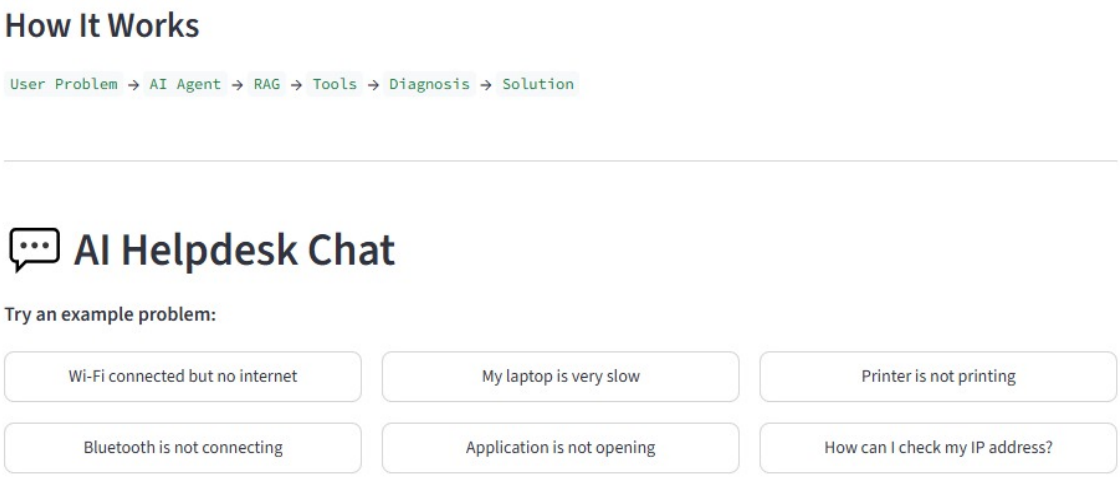
Stage 15 – Displaying the Result

The final answer is displayed to the user through the Streamlit interface.

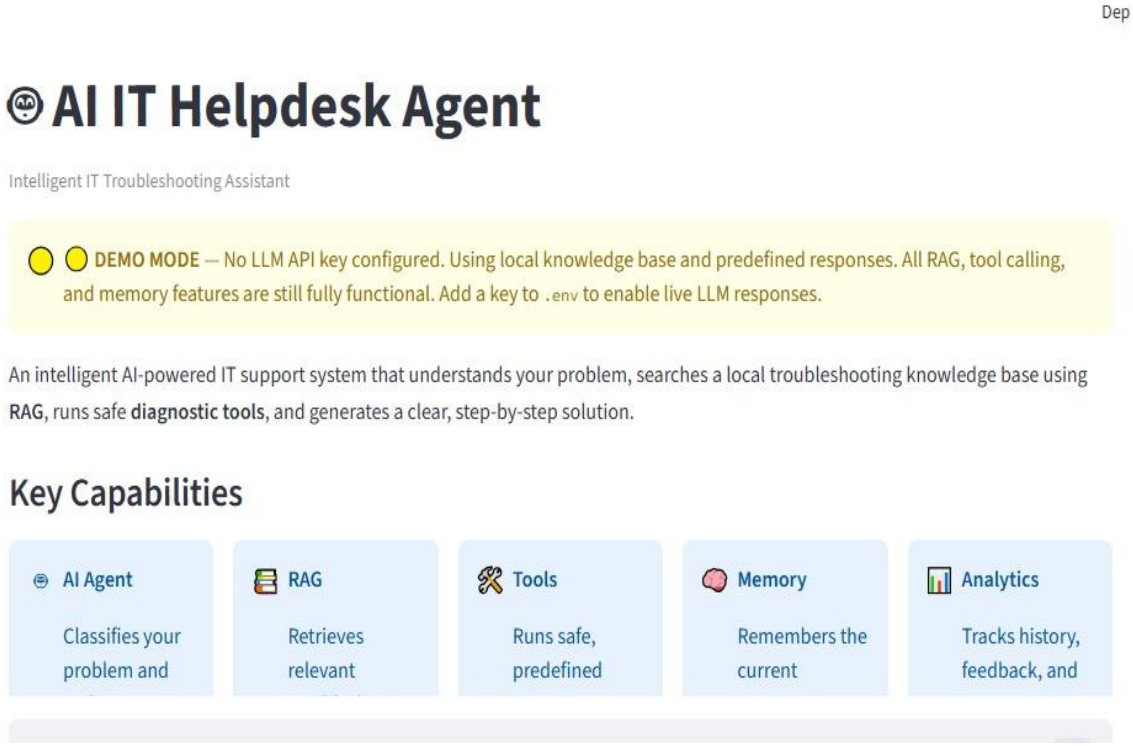
3.3 Screenshots / Output

The following screenshots can be included in the project report:

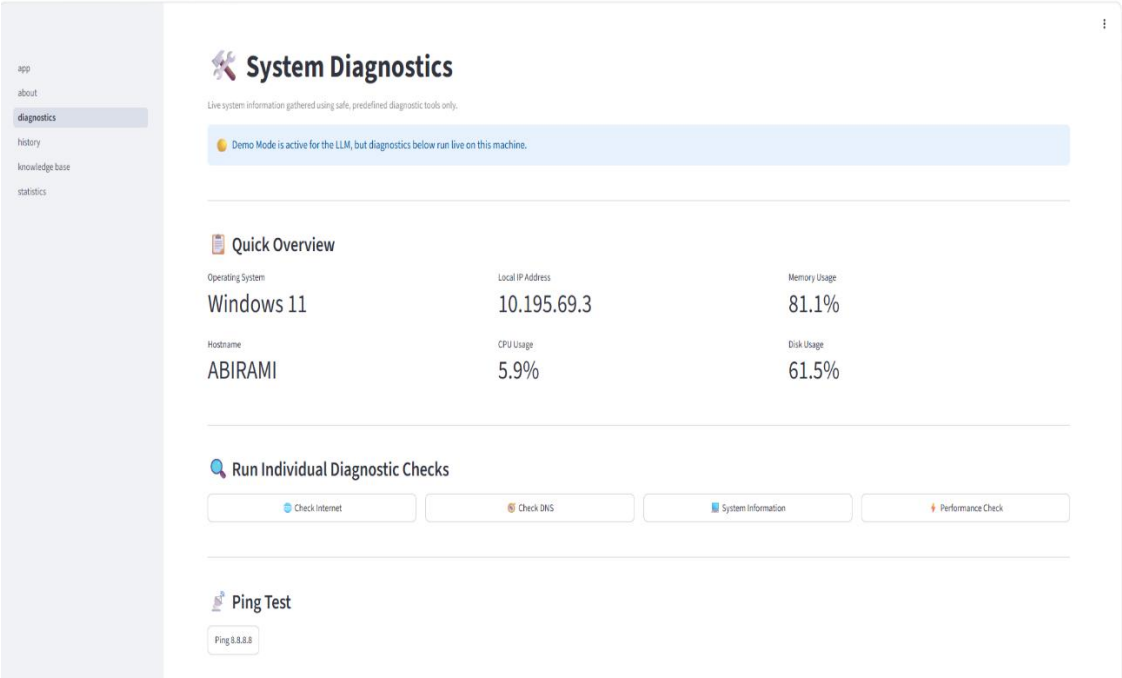
3.3.1.Home Dashboard



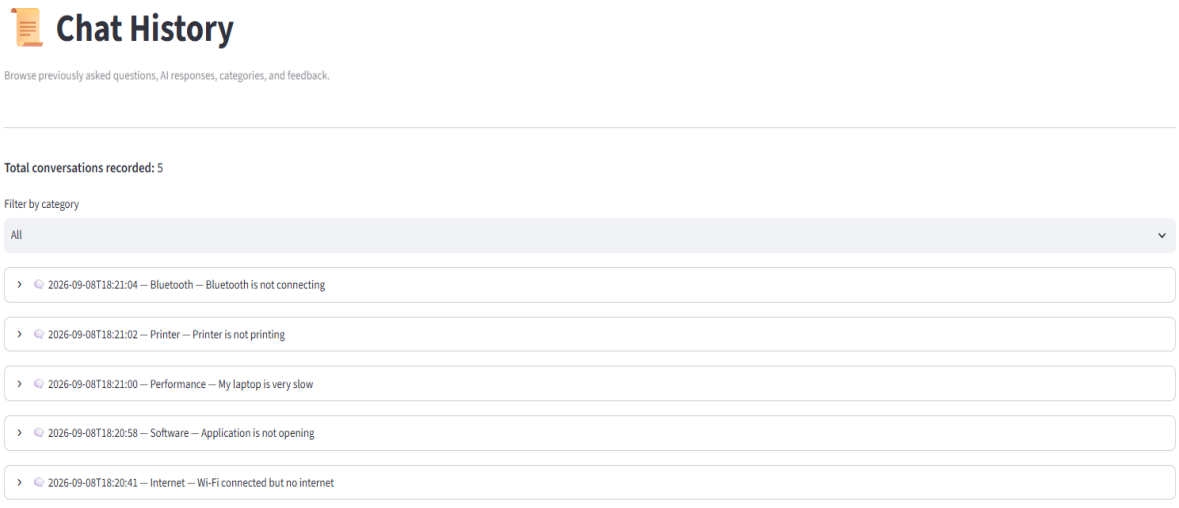
3.3.2.AI IT Helpdesk Chat Interface



3.3.3. System Diagnostics Page



3.3.4.Chat History Page



3.3.5About this project

ℹ️ About This Project

AI IT Helpdesk Agent

Intelligent IT Troubleshooting Assistant

Project Objective

To design and implement an AI-powered IT Helpdesk Agent capable of understanding natural-language descriptions of common computer problems, retrieving relevant troubleshooting knowledge using Retrieval-Augmented Generation (RAG), running safe diagnostic tools, and generating clear, step-by-step solutions — while remembering conversation context and logging feedback for continuous improvement.

Technologies Used

- **Language:** Python 3.11+
- **Frontend:** Streamlit
- **AI / LLM:** Configurable LLM provider (Anthropic Claude or OpenAI), with a fully functional local Demo Mode when no API key is present
- **RAG:** Sentence Transformers for embeddings, ChromaDB for vector storage and similarity search
- **Database:** SQLite for chat history and feedback
- **Diagnostics:** `psutil`, `socket`, `platform`, and a restricted `subprocess` call (ping only, hardcoded arguments)
- **Configuration:** `python-dotenv`

Main Features

1. AI Helpdesk Chat with structured, step-by-step responses
2. RAG-based knowledge base search across 10 IT topics
3. AI Agent layer for intent classification and orchestration
4. Safe, predefined diagnostic tool calling
5. Conversation memory for context-aware follow-ups
6. SQLite-backed chat history
7. User feedback capture (👍 / 👎)
8. System Diagnostics dashboard
9. Knowledge Base management (view / add / delete / rebuild index)
10. Statistics dashboard with category and feedback breakdowns

3.3.6. Statistics Dashboard

📊 Statistics

Project usage statistics: query volume, feedback, and problem category breakdown.

Total Queries	Helpful Responses	Not Helpful	Most Common Issue
5	1	0	Software

Problem Category Breakdown



category	count
Software	1
Printer	1

3.4 Results Achieved

The project successfully demonstrates an AI-based IT troubleshooting and support system.

The following results were achieved.

1. AI-Based IT Troubleshooting

Users can describe common technical problems using natural language.

2. Problem Classification

The system can categorize technical questions into relevant IT categories.

3. Knowledge Retrieval

The RAG system retrieves relevant troubleshooting information from the local knowledge base.

4. Semantic Search

The system can find relevant information based on semantic similarity.

5. AI Agent Decision Making

The Agent determines the required processing and whether a diagnostic tool is useful.

6. Safe Diagnostic Tools

The system can perform predefined diagnostic operations such as checking CPU, memory, disk, IP, DNS, and internet connectivity.

7. Context-Based Responses

Conversation memory allows the system to understand follow-up questions.

8. Clear Troubleshooting Steps

The system provides structured troubleshooting instructions.

9. Chat History

User interactions can be stored and viewed through the history page.

10. Feedback Collection

The system records helpful and not-helpful feedback.

11. Statistics

The system provides statistics about user questions and problem categories.

12. Error Handling

The system handles missing API keys, tool failures, database errors, invalid input, and other runtime problems.

13. Demo Mode

The application can continue to demonstrate common IT troubleshooting scenarios even without an external LLM API key.

14. Secure Configuration

API credentials are kept outside the source code using environment variables.

15. Simple User Interface

The Streamlit interface provides an easy-to-use chat-based experience.

4. CONCLUSION & FUTURE SCOPE

4.1 Project Conclusion

The **AI IT Helpdesk Agent** is a practical Artificial Intelligence application developed to simplify the process of solving common technical problems.

The project combines several modern technologies including:

- Artificial Intelligence
- Large Language Models
- Agentic AI
- Retrieval Augmented Generation
- Vector Databases
- Semantic Search
- Tool Calling
- System Diagnostics

- Conversation Memory
- SQLite Database
- Streamlit
- Python

The main purpose of the project is to provide users with quick and understandable IT support.

Instead of manually searching through websites and troubleshooting documents, users can simply describe their problem using natural language.

The AI Agent analyzes the user's problem and identifies the appropriate category.

The RAG system retrieves relevant troubleshooting information from the knowledge base. The Agent can also use safe diagnostic tools when additional system information is required.

The retrieved knowledge, diagnostic information, and conversation context are combined to generate a final troubleshooting response.

The system also maintains conversation memory so that follow-up questions can be understood.

SQLite is used to store chat history and feedback, while the statistics module provides useful information about system usage.

Security is considered by restricting tool execution to predefined diagnostic functions. The system does not allow arbitrary shell or PowerShell commands.

The project therefore demonstrates the practical combination of **Agent + RAG + Tools + Memory** for an IT support use case.

Overall, the **AI IT Helpdesk Agent** provides a strong foundation for developing a more advanced automated IT support platform in the future.

4.2 Challenges Faced

Several challenges were encountered during the development of the project.

1. Understanding RAG

Understanding how document retrieval and AI generation work together was an important challenge.

2. Knowledge Base Preparation

Creating useful and properly structured troubleshooting documents was necessary for good retrieval results.

3. Text Chunking

Choosing a suitable chunk size is important for retrieval quality.

If chunks are too large, irrelevant information may be retrieved.

If chunks are too small, important troubleshooting context may be lost.

4. Embedding Generation

Understanding embeddings and semantic similarity was another challenge.

5. ChromaDB Integration

Connecting the knowledge base with the vector database required careful handling of embeddings, documents, and metadata.

6. LLM API Integration

Connecting the application with an external LLM API and handling API failures required proper configuration.

7. Preventing Hallucination

AI models may sometimes generate information that is not available in the knowledge base.

The system therefore provides retrieved knowledge as context and instructs the model to avoid unsupported technical claims.

8. Tool Routing

The system needs to determine when a diagnostic tool should be used.

9. Safe Tool Execution

The project must prevent users from executing arbitrary system commands.

Only predefined diagnostic functions are allowed.

10. Conversation Memory

The system needs to maintain recent conversation context while keeping different sessions separate.

11. Database Management

Questions, responses, categories, timestamps, and feedback need to be stored correctly in SQLite.

12. Error Handling

Different errors such as API failures, database failures, tool failures, invalid input, and missing configuration need to be handled without crashing the application.

13. Demo Mode

The system needs to provide useful predefined responses when an external LLM API key is not available.

14. User Interface Design

The application needs to present technical information in a simple and understandable format.

15. System Compatibility

Diagnostic operations need to work reliably on Windows systems without performing unsafe operations.

4.3 Future Enhancements

The project can be improved significantly in the future.

1. Advanced Agentic AI

The Agent can be enhanced to dynamically select multiple tools depending on the user's problem.

2. More Diagnostic Tools

Additional safe tools can be introduced for:

- Network adapter status
- Wi-Fi signal information
- Battery health
- Running processes
- Network configuration
- Windows services
- Hardware information
- Application status

3. Automated Troubleshooting

Future versions can perform more automated and safe troubleshooting actions after receiving user permission.

4. Multilingual Support

Tamil and other Indian languages can be added so that users can communicate with the system more easily.

5. Voice Input

Speech recognition can be added so users can describe technical problems using their voice.

6. Voice Output

Text-to-speech technology can be added so the system can read troubleshooting instructions aloud.

7. Advanced RAG

The RAG system can be improved using advanced retrieval and re-ranking techniques.

8. Persistent Memory

Conversation memory can be expanded to maintain longer-term user interaction history.

9. User Authentication

A secure login system can be added for individual users.

10. Personalized IT Support

The system can provide personalized recommendations based on the user's computer environment and previous support interactions.

11. Admin Dashboard

An administrator dashboard can be developed for:

- Adding knowledge documents
- Updating documents
- Removing outdated documents
- Monitoring system usage

- Managing users
- Viewing common problems

12. Automatic Knowledge Base Updates

The system can automatically detect new troubleshooting documents and update the vector database.

13. Cloud Deployment

The application can be deployed to a cloud platform so users can access the helpdesk from different locations.

14. Mobile Application

A mobile application can be developed for Android and iOS.

15. Advanced Analytics

The system can analyze:

- Most common IT problems
- Frequently used troubleshooting solutions
- Helpful response percentage
- User feedback
- Common categories

16. Improved Security

Future versions can implement:

- Authentication
- Authorization
- Rate limiting
- Secure API management
- Input validation
- Logging
- Monitoring

17. Automatic Problem Detection

The system could automatically identify hardware or software problems using system monitoring information.

18. Ticket Management

A future version can create IT support tickets automatically when an issue cannot be solved by the AI Agent.

19. Human Helpdesk Integration

The system can transfer complex issues to a human IT technician along with the complete conversation and diagnostic information.

20. 24/7 IT Assistance

Once deployed on a suitable server, the AI Helpdesk Agent can provide automated IT support at any time.

4.4 References

1. Python Documentation – Reference for Python programming language and standard libraries.
2. Streamlit Documentation – Reference for developing interactive Python-based web applications.
3. Sentence Transformers Documentation – Reference for generating sentence embeddings and semantic similarity.
4. ChromaDB Documentation – Reference for vector databases, embeddings, collections, and semantic retrieval.
5. SQLite Documentation – Reference for lightweight relational database management.
6. psutil Documentation – Reference for system and process monitoring in Python.
7. python-dotenv Documentation – Reference for managing environment variables in Python applications.
8. Large Language Model API Documentation – Reference for integrating AI-based natural-language generation.
9. Retrieval Augmented Generation – Reference for combining information retrieval with language model generation.
10. Agentic AI Concepts – Reference for AI systems that can reason, select actions, and use tools to accomplish tasks.